Arquitetura de Computadores

Introdução

Quelita Ribeiro

Departamento de Computação / UFS

Introdução

- O que é um computador?

Introdução

- O que é um computador?
  - Sistema natural ou artificial que resolve problemas dinamicamente
  - Atendendo a um conjunto de regras
  - Executando um conjunto de passos

Introdução

- O que é um programa?

Introdução

- O que é um programa?
  - *Uma sequência de instruções descrevendo como realizar determinada tarefa é chamada de* **programa**.

Introdução

- O que é hardware?

Introdução

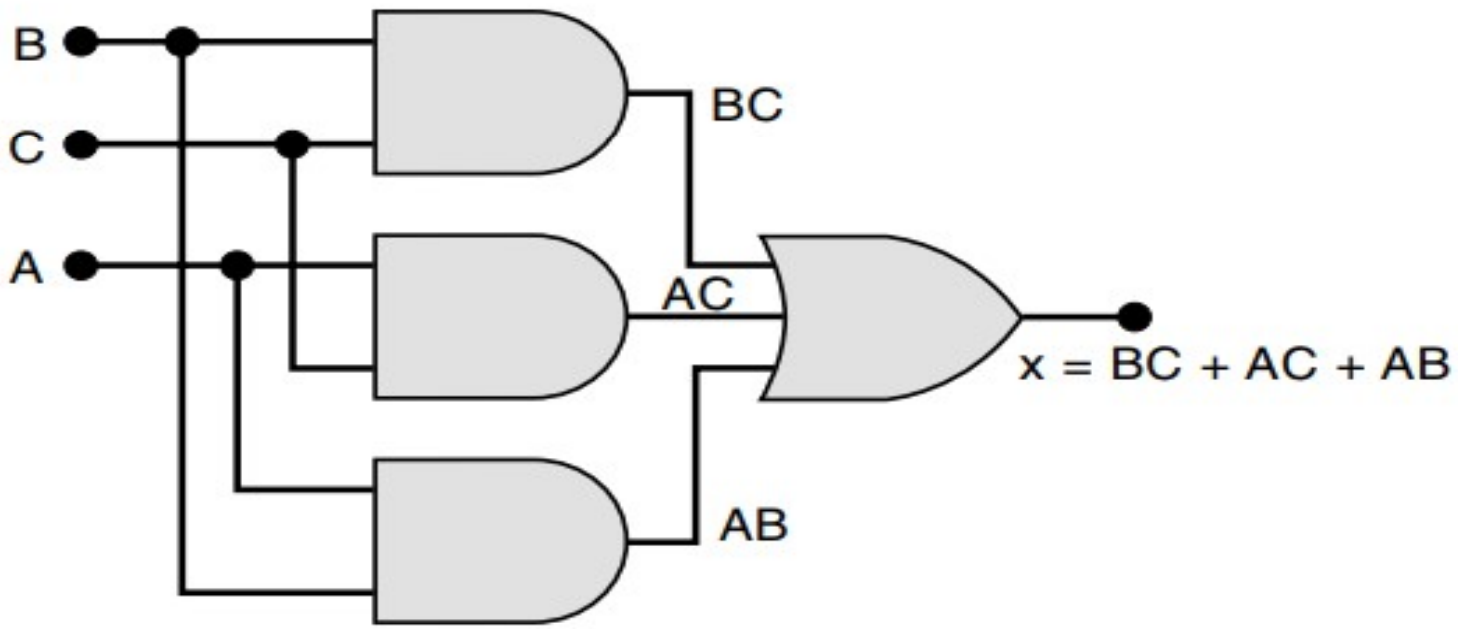
- O que é hardware?
  - Consiste em objetos tangíveis, tais como:
    - circuitos integrados,
    - placas de circuito impresso,
    - cabos,
    - fontes de alimentação,
    - memórias
    - impressoras e outros
  - em vez de algoritmos ou instruções.

Organização estruturada de computadores

Linguagens, níveis e máquinas virtuais

Introdução

- Os circuitos eletrônicos do computador podem reconhecer e executar diretamente um conjunto limitado de instruções simples,



Introdução

- Todos os programas devem ser convertidos antes que possam ser executados.
- Essas instruções básicas não são mais complicadas do que:
    - Some dois números.
    - verifique se um número é zero.
    - Copie dados de uma parte da memória do computador para outra.

Introdução

- O que é linguagem de máquina?

Introdução

- O que é linguagem de máquina?
  - São as instruções primitivas de um computador formam uma linguagem com a qual as pessoas podem se comunicar com o computador.
  - Essa linguagem é denominada **linguagem de máquina**.

Introdução

- Existe uma grande lacuna entre o que é conveniente para as pessoas e o que é conveniente para computadores.

Por quê?

Exemplo

(c) Programa IJVM em hexadecimal.

```
0x15 0x02
0x15 0x03
0x60
0x36 0x01
0x15 0x01
0x10 0x03
0x9F 0x00 0x0D
0x15 0x02
0x10 0x01
0x64
0x36 0x02
0xA7 0x00 0x07
0x10 0x00
0x36 0x03
```

(c)

Nível ISA

Figura 4.14 (a) Fragmento em Java. (b) Linguagem de montagem Java correspondente. (c) Programa IJVM em hexadecimal.

```
i = j + k;
if (i == 3)
    k = 0;
else
   j = j – 1;
```

(a)

```
 1        ILOAD  j          // i = j + k
 2        ILOAD  k
 3        IADD
 4        ISTORE i
 5        ILOAD  i          // if (i == 3)
 6        BIPUSH 3
 7        IF_ICMPEQ L1
 8        ILOAD j           // j = j – 1
 9        BIPUSH 1
10        ISUB
11        ISTORE j
12        GOTO  L2
13   L1:  BIPUSH  0         // k = 0
14        ISTORE  k
15   L2:
```

(b)

```
0x15  0x02
0x15  0x03
0x60
0x36  0x01
0x15  0x01
0x10  0x03
0x9F  0x00  0x0D
0x15  0x02
0x10  0x01
0x64
0x36  0x02
0xA7  0x00  0x07
0x10  0x00
0x36  0x03
```

(c)

Introdução

- **O problema de comunicação com o computador pode ser abordado de duas maneiras**

- Ambas envolvem projetar **um novo conjunto de instruções**

- Pois é mais conveniente para as pessoas usarem do que o conjunto embutido de instruções de máquina.

Introdução

- As **instruções de máquina embutidas** formam uma linguagem, que chamaremos de **L0**.

- Novas instruções formam uma linguagem, que chamaremos de **L1**,

- As duas técnicas em L1 diferem no modo como os programas escritos em **L1** são **executados** pelo computador que, afinal, só pode executar programas escritos em sua linguagem de máquina, **L0**.

Introdução

- Um método de execução de um programa escrito em **L1** é:

- Primeiro substituir cada instrução nele por uma sequência equivalente de instruções em **L0**.

- O programa resultante consiste totalmente em instruções **L0**.

- ***O computador, então, executa o novo programa L0 em vez do antigo programa L1.***

Qual o nome dessa técnica?

Introdução

Qual o nome dessa técnica?

Essa técnica é chamada de **tradução.**

Introdução

- A outra técnica é escrever um programa que:

- Considere os programas em **L1** como dados de entrada e os execute, examinando cada instrução por sua vez

- **Executando diretamente a sequência equivalente de instruções L0.**

- Essa técnica não requer que se gere um novo programa em **L0.**

Qual o nome dessa técnica?

Introdução

- A técnica é chamada de **interpretação**
- O programa que a executa é chamado de **interpretador.**

Introdução

- Tradução e interpretação são semelhantes.
- Nos dois métodos, o computador executa instruções em L1 executando sequências de instruções equivalentes em L0.

Qual a diferença?

Introdução

- Qual a diferença?
- A diferença é que:
  - Na tradução, o programa L1 inteiro primeiro é convertido para um L0,
  - O programa L1 é desconsiderado e depois o novo L0 é carregado na memória do computador e executado.
  - Durante a execução, o programa L0 recém-gerado está sendo executado e está no controle do computador.

Introdução

- Qual a diferença?
- A diferença é que:
  - Na interpretação, depois que cada instrução L1 é examinada e decodificada, ela é executada de imediato.
  - **Nenhum programa traduzido é gerado.**
  - O interpretador está no controle do computador.
  - Para ele, o programa L1 é apenas dados.
- Ambos os métodos são bastante utilizados.

Introdução

Agora, em vez de pensar em termos de tradução ou interpretação, muitas vezes é simples imaginar a existência de um computador hipotético ou **máquina virtual** cuja linguagem seja **L1**.

Introdução

- Vamos chamar essa máquina virtual de M1 *(e de M0 aquela correspondente a L0)*.
- Se essa máquina pudesse ser construída de forma barata, não seria preciso ter a linguagem **L0** ou uma máquina que execute os programas em **L0**.

Será que hoje em dia já é possível?

Introdução

- As pessoas poderiam escrever seus programas em L1
- Fazer com que o computador os executasse diretamente.
- Mesmo que a máquina virtual cuja linguagem é L1 seja muito cara ou complicada de construir com circuitos eletrônicos, as pessoas ainda podem escrever programas para ela.

Introdução

- Esses programas podem ser ou interpretados ou traduzidos por um programa escrito em L0 que, por si só, consegue ser executado diretamente pelo computador real.
- **Em outras palavras, as pessoas podem escrever programas para máquinas virtuais, como se realmente existissem.**

Introdução

- Para tornar prática a tradução ou a interpretação, as linguagens L0 e L1 não deverão ser “muito” diferentes.

- Tal restrição significa que L1, embora melhor que L0, ainda estará longe do ideal para a maioria das aplicações.

Introdução

- Esse resultado talvez seja desanimador à luz do propósito original da criação de L1:

- aliviar o trabalho do programador de ter que expressar algoritmos em uma linguagem mais adequada a máquinas do que a pessoas.

Introdução

- Porém, a abordagem óbvia é inventar outro conjunto de instruções que seja mais orientado a pessoas e menos orientado a máquinas que a L1.

Introdução

- Esse terceiro conjunto também forma uma linguagem, que chamaremos de L2 (e com a máquina virtual M2).

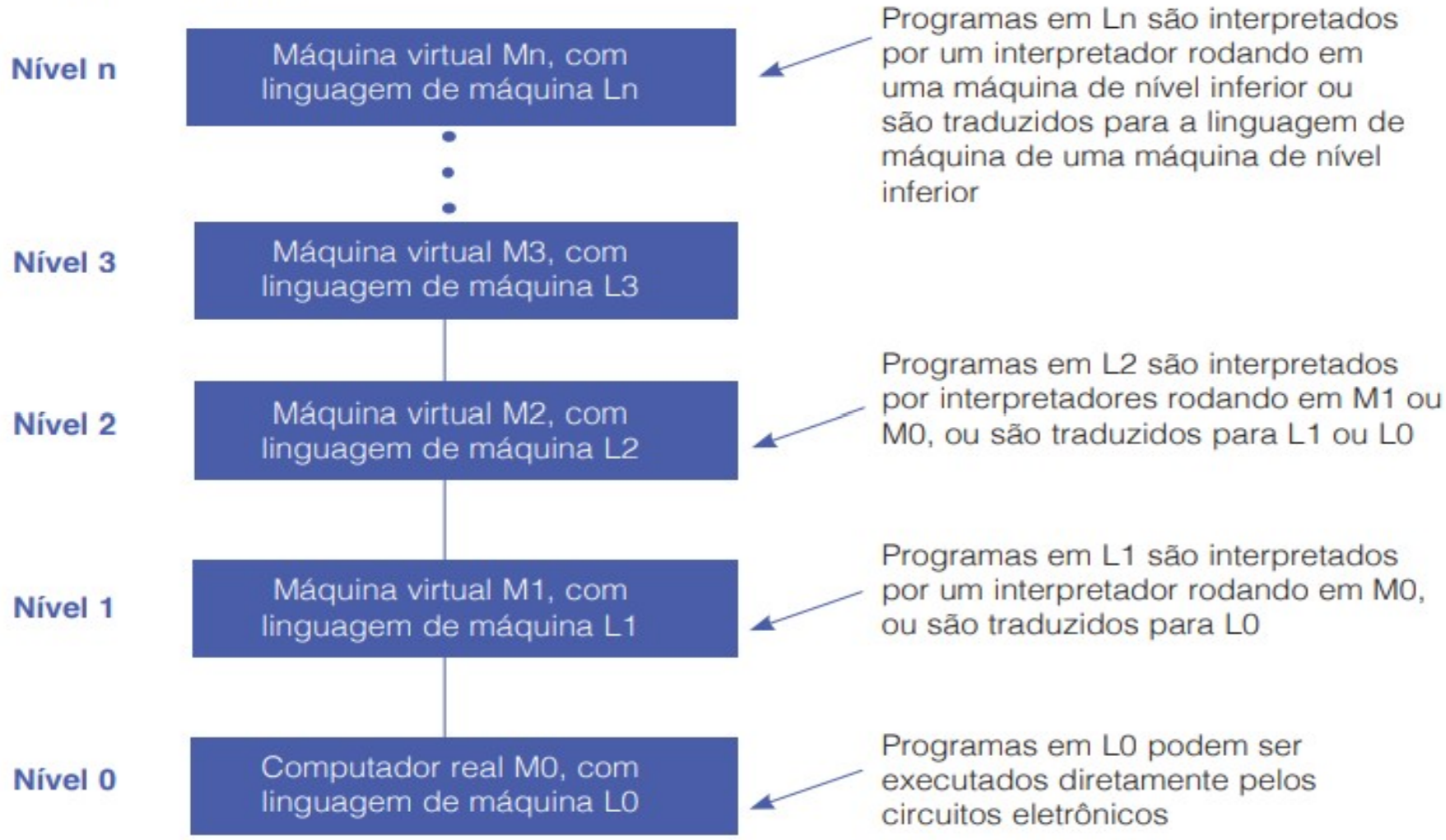
- As pessoas podem escrever programas em L2 exatamente como se de fato existisse uma máquina real com linguagem de máquina L2.

- Esses programas podem ser traduzidos para L1 ou executados por um interpretador escrito em L1.

Introdução

Cada linguagem usa sua antecessora como base, portanto, podemos considerar um computador que use essa técnica como uma série de camadas ou níveis, um sobre o outro, conforme mostra a Figura 1.1.

Figura 1.1 Máquina multinível.



Introdução

- Um computador com n níveis pode ser visto como n diferentes máquinas virtuais, cada uma com uma linguagem de máquina diferente.

- Apenas programas escritos na linguagem L0 podem ser executados diretamente pelos circuitos eletrônicos, sem a necessidade de uma tradução ou interpretação intervenientes.

Introdução

- Os programas escritos em L1, L2, ..., Ln devem ser interpretados por um interpretador rodando em um nível mais baixo ou traduzidos para outra linguagem correspondente a um nível mais baixo.

Exercício

1) A linguagem Java é interpretada ou traduzida? Explique.
2) A linguagem C é interpretada ou traduzida? Explique.
3) A linguagem Fortran é interpretada ou traduzida? Explique.

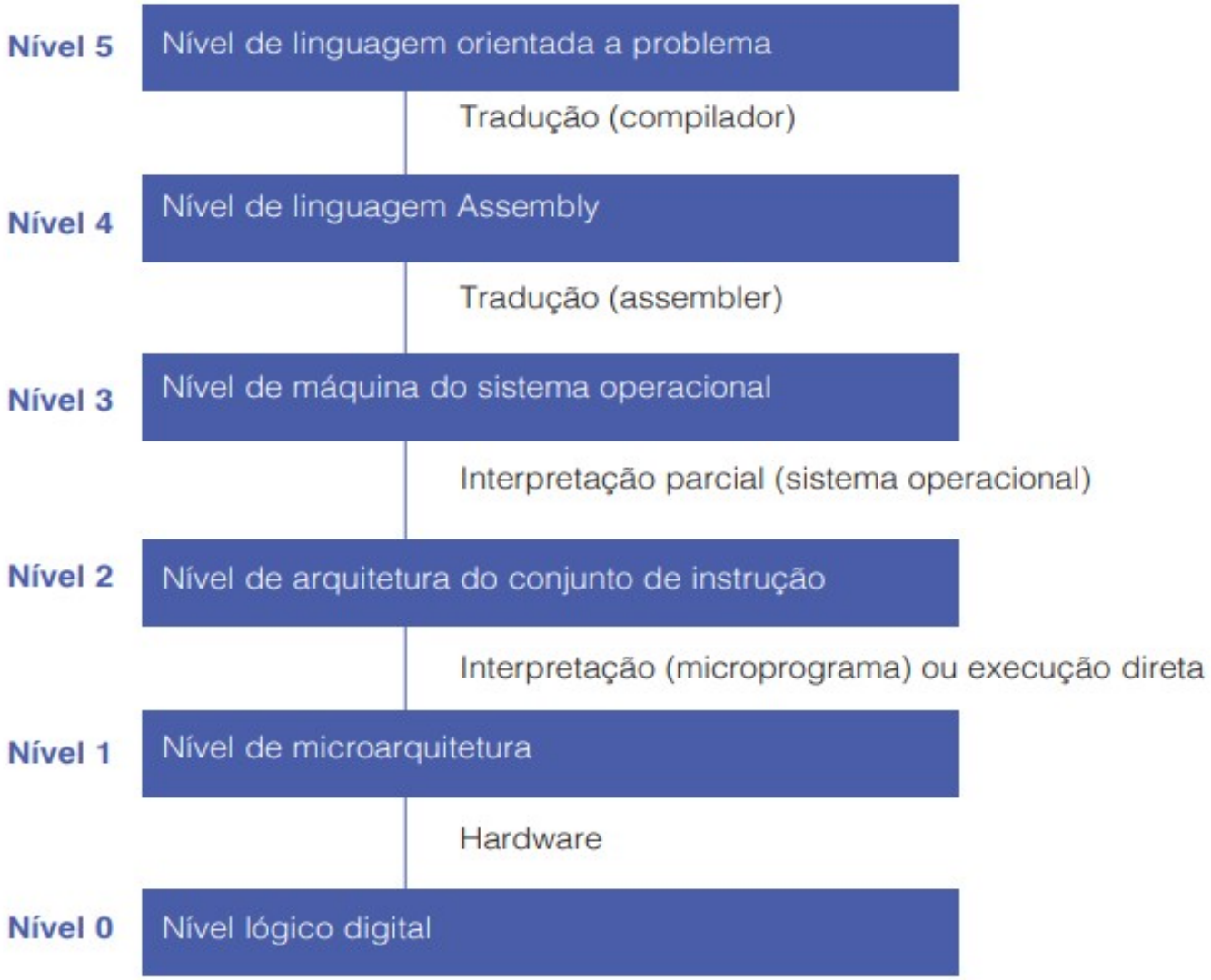
Organização estruturada de computadores

Máquinas multiníveis contemporâneas

Introdução

A maioria dos computadores consiste de dois ou mais níveis. Existem máquinas com até seis níveis, conforme mostra a Figura

Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- O nível 0, na parte inferior, é o hardware verdadeiro da máquina.
    - Seus circuitos executam os programas em linguagem de máquina do nível 1.
- No nível mais baixo, o nível lógico digital, os objetos interessantes são chamados de portas (ou gates).

Introdução

- Embora montadas a partir de componentes analógicos, como transistores, podem ser modeladas com precisão como dispositivos digitais.

- Cada porta tem uma ou mais entradas digitais (sinais representando 0 ou 1) e calcula como saída alguma função simples dessas entradas, como AND (E) ou OR (OU).

Introdução

- Um pequeno número de portas podem ser combinadas para formar uma memória de 1 bit, que consegue armazenar um 0 ou um 1.

- As memórias de 1 bit podem ser combinadas em grupos de (por exemplo) 16, 32 ou 64 para formar registradores.

Introdução

- Cada registrador pode manter um único número binário.

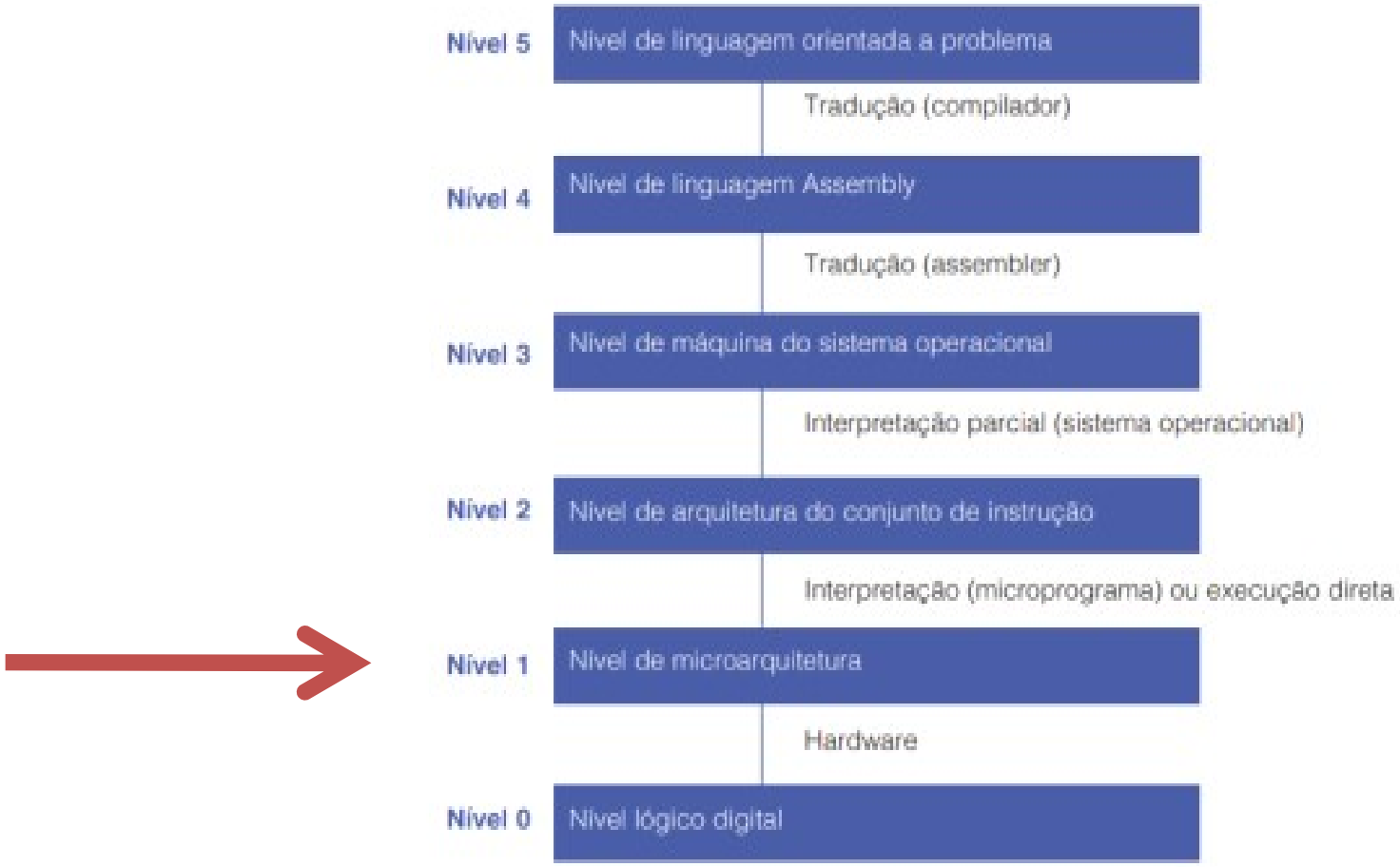
Será? O que temos de novo?

- As portas também podem ser combinadas para formar o próprio mecanismo de computação principal.

Introdução

- O próximo nível acima é o nível de **microarquitetura**.

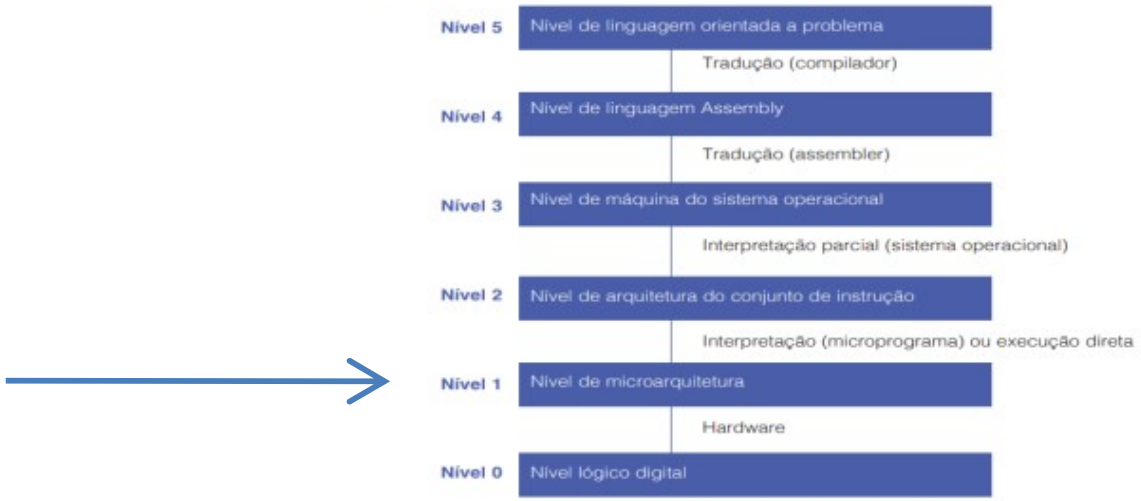
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- Podemos ter uma coleção de 8 a 32 registradores que formam uma memória local e um circuito chamado ULA – Unidade Lógica e Artitmética, que é capaz de realizar operações aritméticas simples.

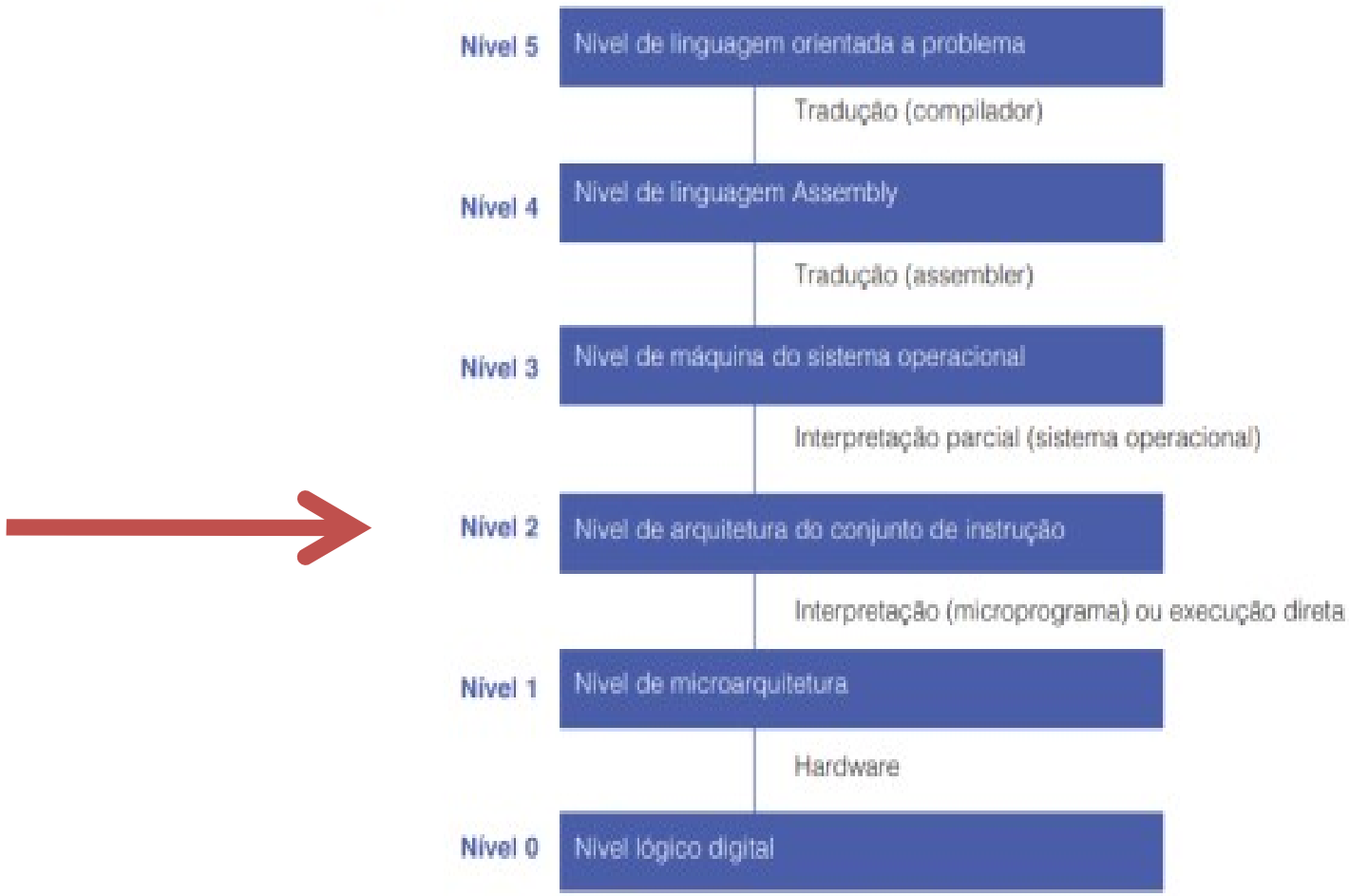
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- O nível 2 é o nível de arquitetura do conjunto de instrução, ou nível ISA (*Instruction Set Architecture*).

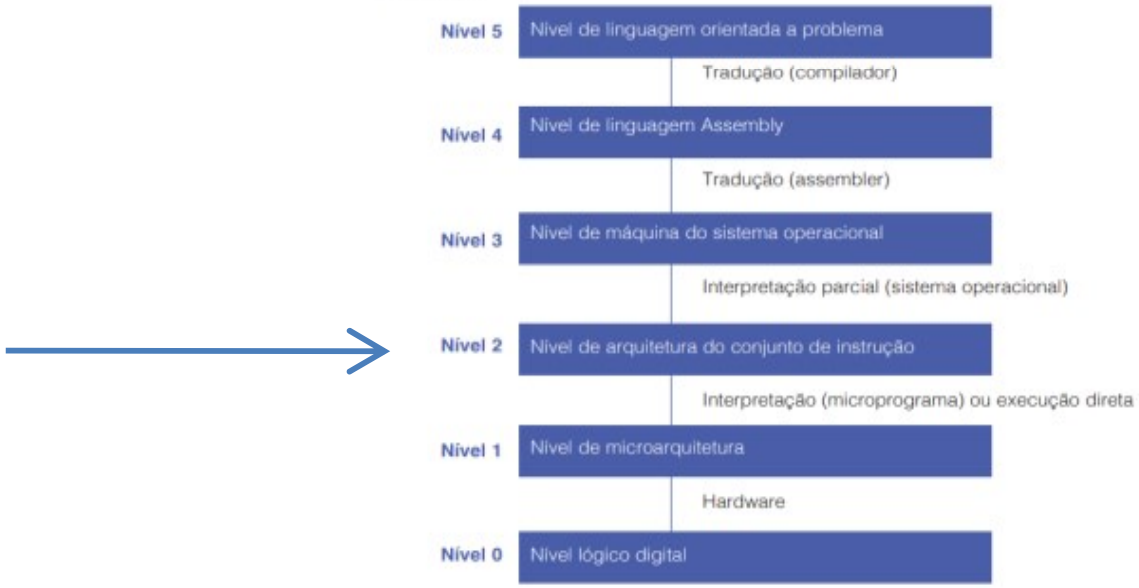
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- Os fabricantes publicam o manual para cada computador que vendem, chamado de "Manual de Referência da Linguagem de Máquina".

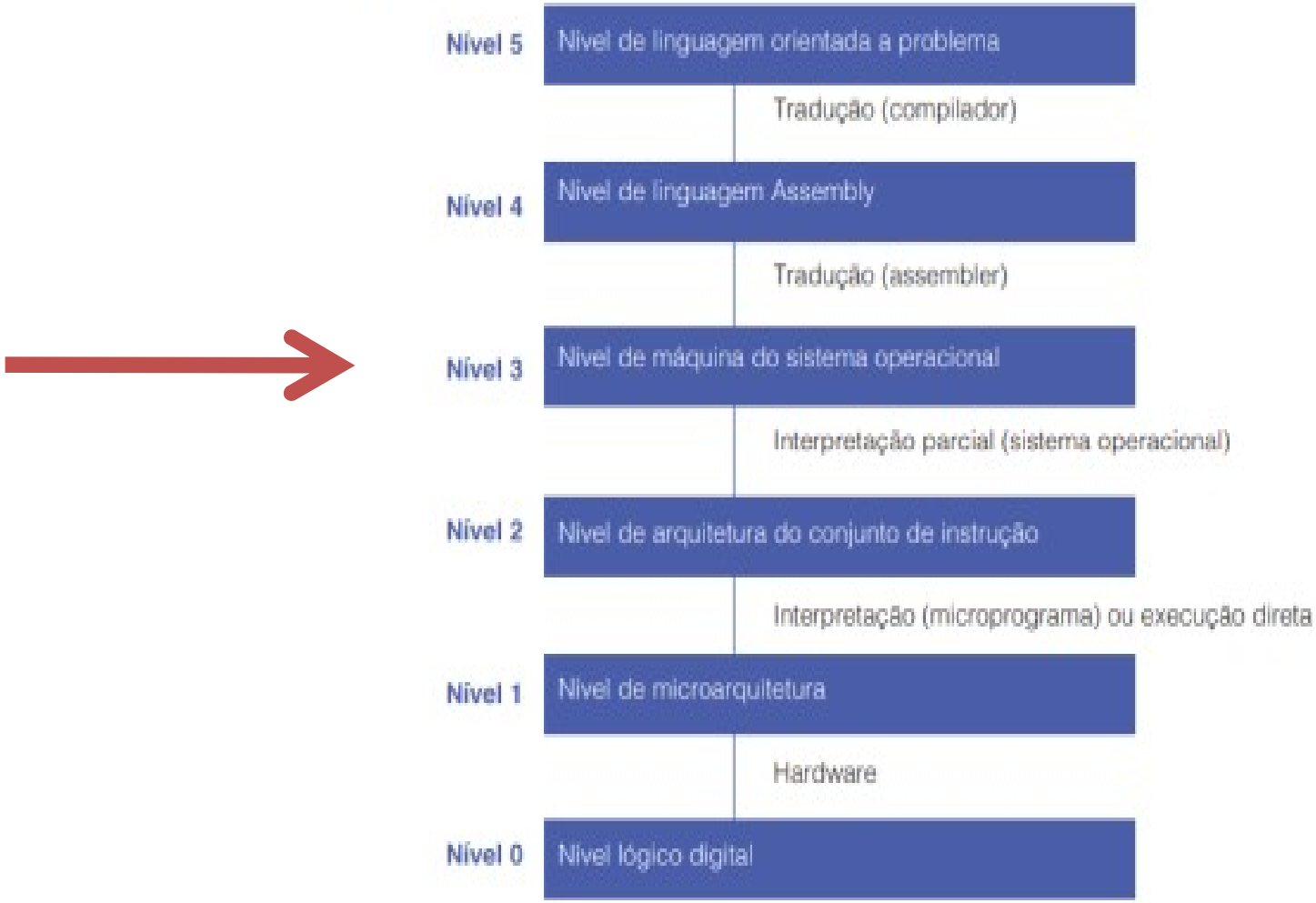
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- O nível 3 costuma ser híbrido.
  - A maior parte das instruções em sua linguagem também está no nível ISA.

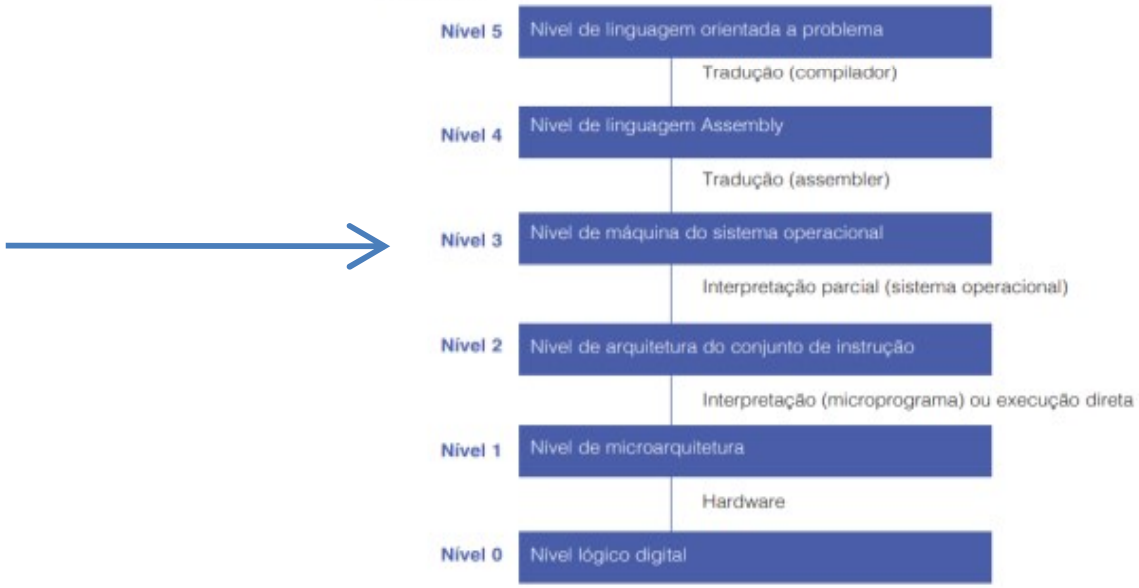
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- O nível 3
- Além disso, há:
  - um conjunto de novas instruções,
  - uma organização de memória diferente,
  - capacidade de executar dois ou mais programas simultaneamente e diversos outros recursos.

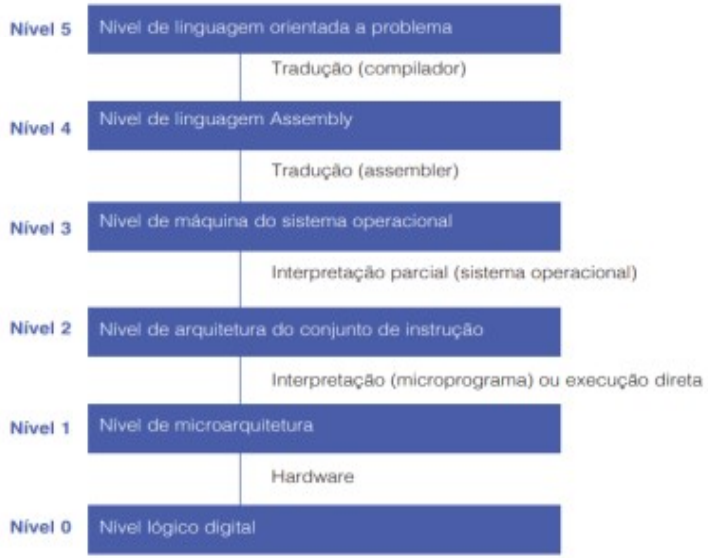
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- Os três níveis mais baixos não são usados pelo programador comum.
- Eles são voltados principalmente para a execução dos interpretadores e tradutores necessários para dar suporte aos níveis mais altos.

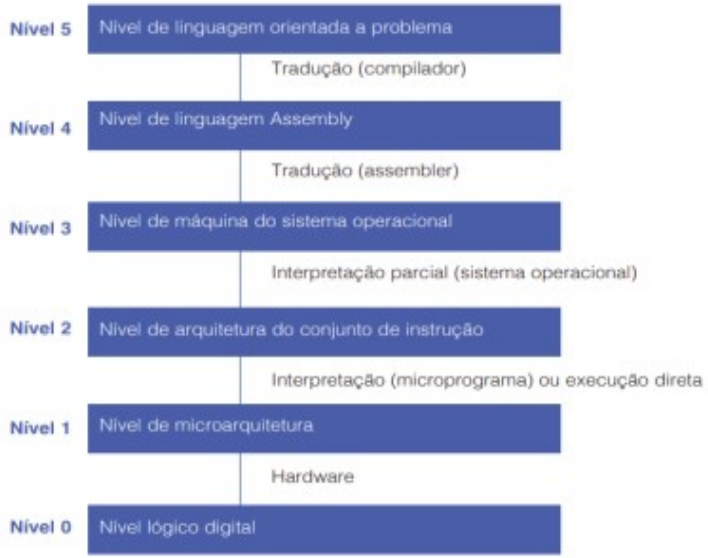
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- Esses interpretadores e tradutores são escritos pelos programadores de sistemas, profissionais que se especializam no projeto e execução de novas máquinas virtuais.

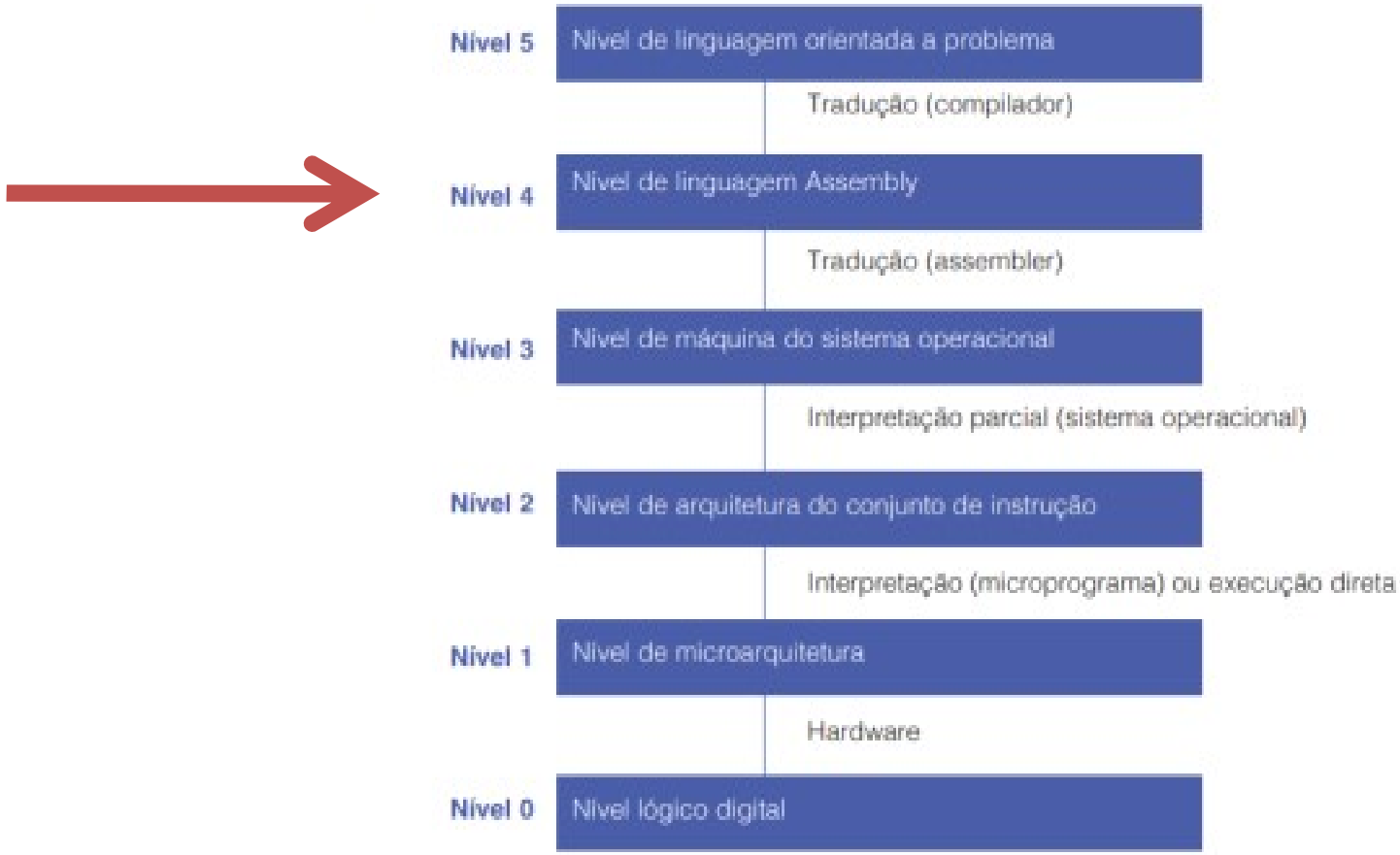
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

Os níveis 4 e acima são voltados para o programador de aplicações, que tem um problema para solucionar

Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).

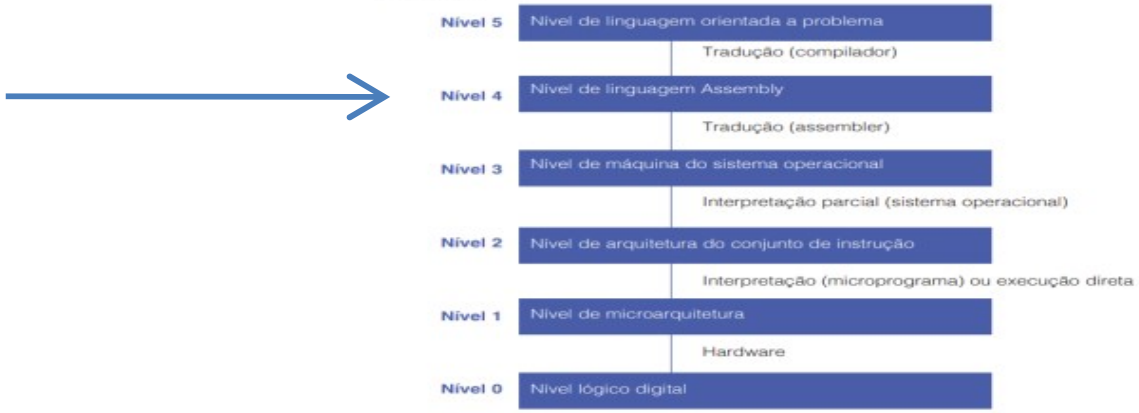


Introdução

- O nível 4 é o nível da linguagem de montagem (assembly).

- Esse nível é uma forma simbólica, pois fornece um método para as pessoas escreverem programas para os níveis 1, 2 e 3.

Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).

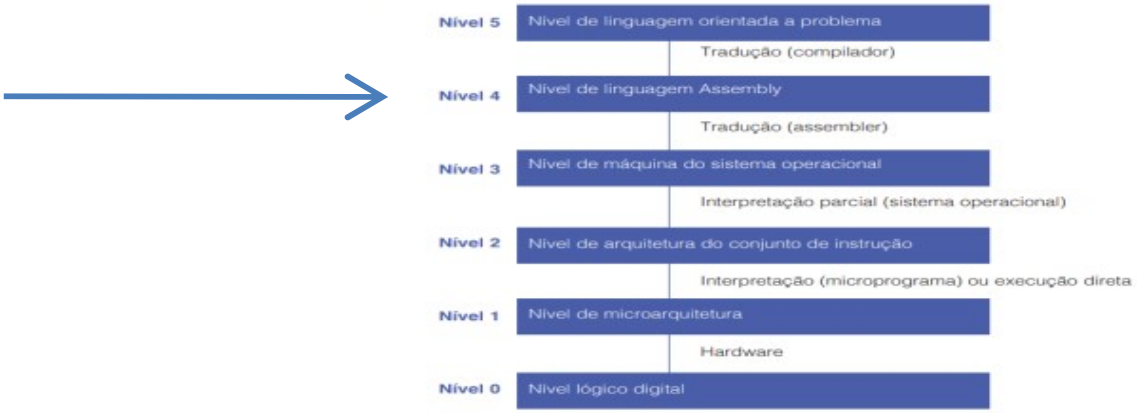


Introdução

- Programas em linguagem de montagem são primeiro traduzidos para linguagem de nível 1, 2 ou 3, e em seguida interpretados pela máquina virtual ou real adequada.

- O programa que realiza a tradução é denominado **assembler**.

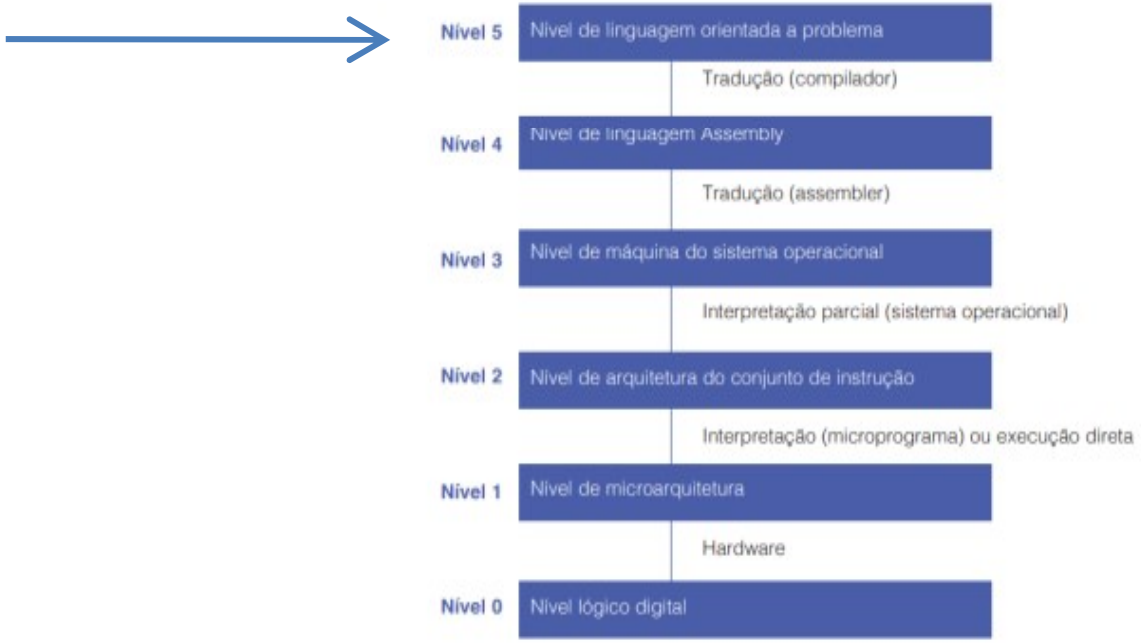
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- O nível 5 consiste em linguagens projetadas para ser usadas por programadores de aplicações.
- Essas linguagens costumam ser denominadas linguagens de alto nível.
- Existem literalmente centenas delas. Algumas das mais conhecidas são C++, Java, Perl, Python e PHP.

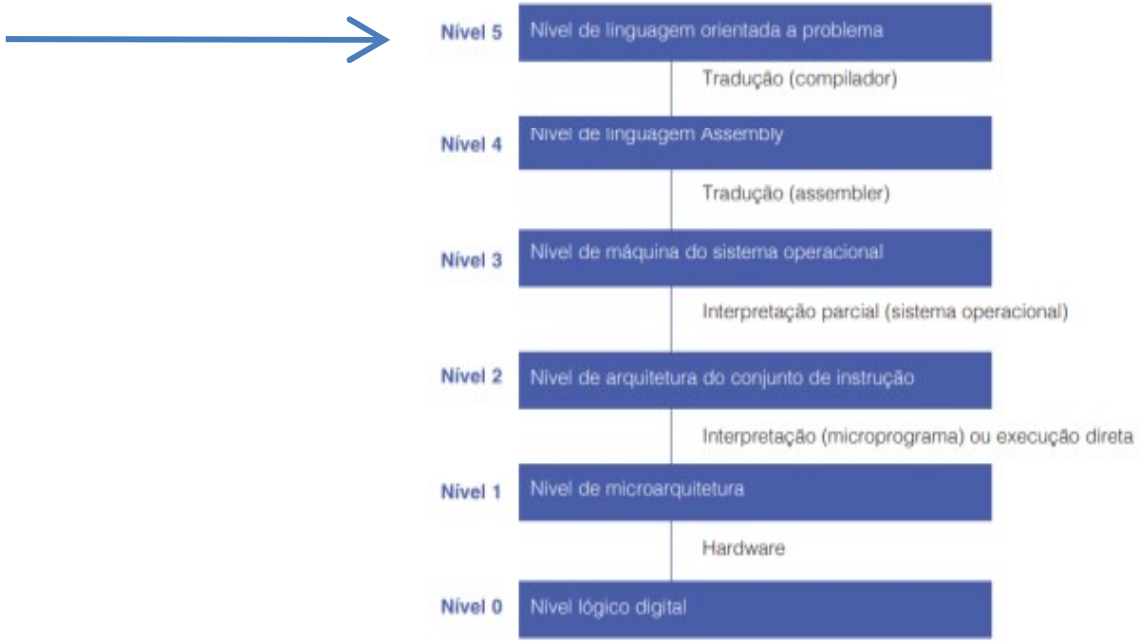
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- Programas escritos nessas linguagens em geral são traduzidos para nível 3 ou nível 4 por:
  - tradutores conhecidos como **compiladores**, embora às vezes sejam **interpretados**, em vez de traduzidos.

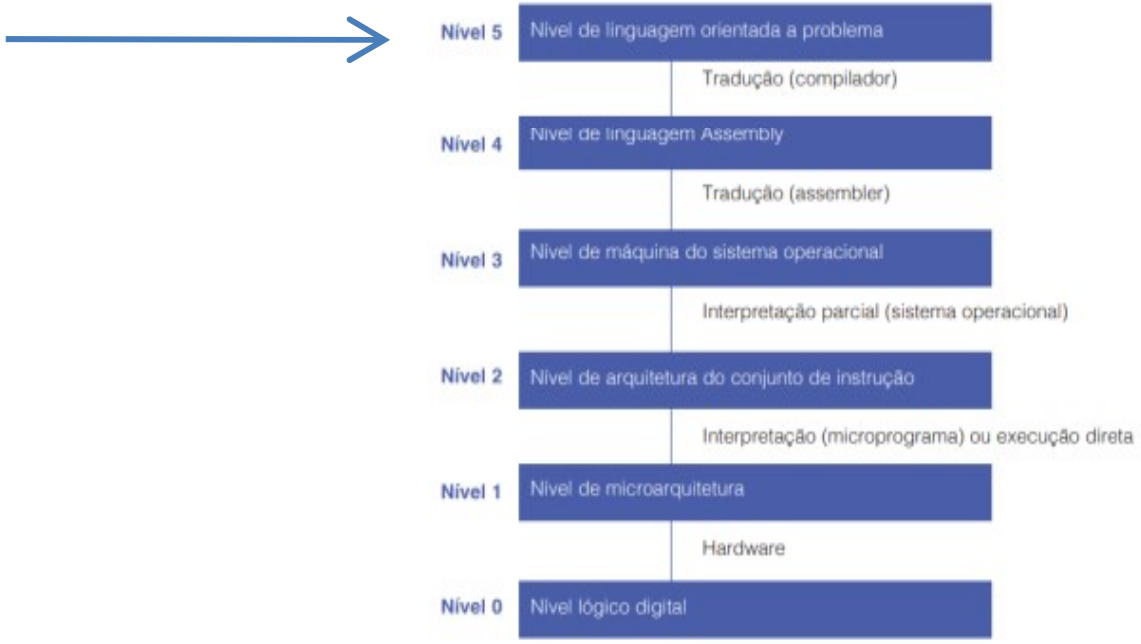
Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

- Programas em Java, por exemplo, costumam ser primeiro traduzidos para uma linguagem semelhante à ISA denominada código de bytes Java, ou bytecode Java, que é então interpretada.

Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Introdução

Resumindo:

- O aspecto fundamental a lembrar é que computadores são projetados como uma série de níveis, cada um construído sobre seus antecessores.
- Cada nível representa uma abstração distinta na qual estão presentes diferentes objetos e operações.
- O conjunto de tipos de dados, operações e características de cada nível é denominado **arquitetura**.
- A **arquitetura** trata dos aspectos que são visíveis ao usuário daquele nível.

Introdução

- Características que o programador vê, como a quantidade de memória disponível, são parte da arquitetura.
- Aspectos de implementação, como o tipo da tecnologia usada para executar a memória, não são parte da arquitetura.
- O estudo sobre como projetar as partes de um sistema de computador que sejam visíveis para os programadores é denominado **arquitetura de computadores.**
- Na prática, arquitetura de computadores e organização de computadores significam basicamente a mesma coisa.